

```

#include <iostream>
int main()
{
    int x = 0;
    int y = 1;
    int z = 2;
    int t = ((z > 0 ? ++x : y++) = z++);
    std::cout << x << y << z << t;
}

```

1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.

2) Ответьте на те же вопросы, если вместо `y++` написать `++y`.

1) Будет CE, т.к. выражение `(z > 0 ? ++x : y++)` - это `rvalue` независимо от значения выражения, и ему нельзя присваивать.

2) Если заменить `y++` на `++y`, то все ОК, и выведется 2132.

Рассмотрим программу:

```
#include <iostream>
int* f(int* p)
{
    *p = 0;
    return p + 2;
}
int* f(int a[5])
{
    a[0] = 0;
    return a + 2;
}
int main()
{
    int a[5] = {1, 2, 3, 4, 5};
    int* p = f(a);
    for (int i = -2; i <= 2; ++i)
    {
        std::cout << p[i];
    }
}
```

- 1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.
- 2) Ответьте на те же вопросы, если полностью удалить вторую версию функции f, оставив только первую.
- 3) Ответьте на те же вопросы, если не только удалить вторую версию функции f, но и вместо f(a) при вызове этой функции написать f(a+1).

- 1) В этой программе CE, т.к. нельзя делать перегрузку функций с аргументами int* и int[5]. Это будет считаться одной и той же функцией, определенной дважды.
- 2) Если удалить вторую версию и оставить только первую, то все ОК, и выведется 02345. Массив неявно приводится к указателю на начальный элемент.
- 3) Если вместо f(a) написать f(a+1), то это скомпилируется, но будет UB. Так как в цикле мы будем обращаться правее правой границы массива. Скорее всего, последним выведется случайное число.

Рассмотрим программу:

```
int main()
{
    int x = 0;
    int& y = x;
    const int& z = x;
    ++y;
    std::cout << x << y << z;
    int& t = z;
    int u = t;
    ++u;
    std::cout << x << y << z << t << u;
}
```

1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.

2) Ответьте на те же вопросы, если вместо **int& t = z;** написать **int& t = y;**

1) Будет CE в строке `int& t = z`, т.к. нельзя под `int&` пытаться подложить `const int`.

2) Если написать **int& t = y**, то все ОК, и выведется 11111112. Потому что `x`, `y`, `z`, `t` - это будут разные имена одной и той же сущности, и лишь `u` будет отличаться.

Рассмотрим программу:

```
#include <iostream>
int sum(int x, int y)
{
    return x + y;
}
double sum(double x, double y)
{
    return x + y + 1;
}
int main()
{
    int c = sum(1, 2);
    double d = sum(3, 4);
    std::cout << c << ' ' << d;
}
```

1) Вызовет ли этот код СЕ, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.

2) Ответьте на те же вопросы, если бы первая функция принимала оба аргумента не `int`, а **`float`**, и возвращаемый тип тоже имела бы не `int`, а **`float`**.

1) Все ок, выведется 3 7. Потому что выбор версии функции происходит по типам принимаемых аргументов, а не результата.

2) Если бы первая версия `sum` принимала `float`, то было бы СЕ. Потому что приведение `int->float` и `int->double` - это приведения одинакового ранга, и между ними нельзя выбрать лучшее, это неоднозначность.

Рассмотрим программу:

```
#include <iostream>
int x = 0;
namespace N
{
    int x = 3;
}
int main()
{
    int x = 1;
    // using N::x;
    {
        int x = 2;
        std::cout << x << ::x;
    }
    std::cout << x;
}
```

1) Вызовет ли этот код СЕ, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.

2) Ответьте на те же вопросы, если раскомментировать строку `using N::x;`

1) Все ок, выведется 201. Выражение `::x` означает обращение к глобальному `x`.

2) Если раскомментировать `using N::x;`, то будет СЕ, т.к. это будет повторное объявление имени `x` в одной и той же области видимости.

Рассмотрим программу:

```
#include <iostream>
class A
{
private:
    int* p = new int;
public:
    A(int x)
    {
        p = new int(x);
    }
    ~A()
    {
        delete p;
    }
    int& getInt()
    {
        return *p;
    }
    A(const A&) = default;
};
int main()
{
    A a(5);
    A b = a;
    std::cout << a.getInt() << b.getInt();
}
```

1) Какая ошибка возникнет, если запустить эту программу, и почему? Дайте развернутое объяснение.

2) Не меняя остальной код, реализуйте конструктор копирования для данного класса таким образом, чтобы код запускался и завершался успешно.

3) Какая еще проблема, кроме только что исправленной, есть в данном коде? Как нужно **исправить конструктор класса A, чтобы устранить эту проблему?**

1) В классе A неявно сгенерится тривиальный копи-конструктор. Когда объект скопируется, у a и b поле p будет одинаковым. Но тогда при выходе из main произойдет ошибка double free, т.к. будет дважды сделано delete по одному адресу. Эта программа упадет с RE.

2) Можно реализовать так:

```
A(const A& other): p(new int(other.p)) {}
```

3) Еще в этом коде есть утечка памяти. Потому что при вызове A(int x) поле p сначала инициализируется через new int, а потом ему присваивается new int(x). Чтобы исправить, надо вместо p = new int(x) использовать список инициализации: **A(int x): p(new int(x)) {}**

Рассмотрим программу:

```
#include <iostream>
int x = 0;
struct S
{
    int* p = new int(0);
    int& a = x;
    void f() /*const*/
    {
        *p = 1;
        a = 2;
        std::cout << *p;
    }
    ~S() { delete p; }
};

int main()
{
    const S s;
    s.f();
    std::cout << x;
}
```

1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.

2) Ответьте на те же вопросы, если раскомментировать слово `const`.

1) Этот код вызывает CE, т.к. мы пытаемся вызвать неконстантный метод `f()` у константного объекта.

2) Если написать `f()` `const`, то все ок и выведется 12. Константность объекта означает константность самих его полей, но не того, на что они ссылаются. При присваивании `a=2` поменяется глобальный `x`, т.к. это была ссылка на него.

Рассмотрим код:

```
#include <iostream>
struct S
{
    int x = 1;
private:
    struct SS
    {
        float y = x;
    };
public:
    SS f()
    {
        SS ss;
        std::cout << ss.y;
        return ss;
    }
};
int main()
{
    S s;
    std::cout << s.f().y;
}
```

1) Из-за чего в этом коде возникает ошибка компиляции? Объясните подробно.
2) Прodelайте в коде минимально возможное количество исправлений, чтобы он начал компилироваться. Ответьте после этого на вопрос, что выведется при запуске этого кода.

1) СЕ, потому что в строке "float y = x;" непонятно что такое x. Ибо x - это нестатическое поле класса S, а у нас нет никакого объекта типа S, откуда бы его можно было взять.

2) Достаточно изменить один символ: вместо float y = x; написать float y = 0;. Тогда скомпилируется и выведет 00.

Скорее всего, будет очень популярный неправильный ответ - будет СЕ из-за приватности, надо убрать приватность. Это неверно. За упоминание этого в случае наличия также и правильного ответа - минус балл.

Рассмотрим код:

```
struct UserId
{
    int id = 0;
    /*explicit*/ UserId(int id): id(id) {} // 1
    /*explicit*/ operator int() // 2
    {
        return id;
    }
};
int operator+(UserId a, UserId b)
{
    return a.id + b.id;
}
int main()
{
    UserId id = UserId(5);
    std::cout << id + 1;
}
```

- 1) Из-за чего в этом коде возникает ошибка компиляции? Объясните развернуто.
- 2) Что изменится, если раскомментировать слово `explicit` в первом случае? А если **раскомментировать оба слова `explicit`**? В каждом случае объясните ответ.

1) СЕ возникает из-за неоднозначности. Выражение `id+1` можно понять как `int(id)+1`, а можно как `id+UserId(1)`. Обе конверсии допустимы неявно, причем у них одинаковый ранг, так что выбрать лучший вариант не получается.

2) Если сделать `explicit UserId(int id)`, то вариант `id+UserId(1)` отпадет и останется только `int(id)+1`, все будет ок. Если же оба раза написать `explicit`, то оба варианта станут недопустимыми и снова будет СЕ.

Рассмотрим программу:

```
#include <iostream>
struct Granny
{
    //friend struct Son;
    int x = 0;
    void f() {}
};
struct Mom: private Granny
{
    int y = 1;
protected:
    void g()
    {
        std::cout << x;
        f();
    }
};
struct Son: public Mom
{
    int z = 2;
    void h()
    {
        std::cout << y;
        f();
    }
};
int main()
{
    Son s;
}
```

1) Скомпилируется ли эта программа? Если да, то просто напишите “да”, а если нет, то укажите все места, в которых возникнут ошибки компиляции, и почему они возникнут.

2) Что изменится, если раскомментировать строчку friend struct Son? Вновь объясните ответ.

1) Ошибка будет только в функции h() при попытке вызвать оттуда f(). Потому что f() - это метод Granny, а Mom скрывает от всех (включая Son) часть, унаследованную от Granny. В остальных местах все ок. Балл за этот пункт ставится, если правильно указана эта ошибка и не указано лишних!

2) Если написать friend struct Son в Granny, то СЕ не исчезнет, т.к. это не Granny, а Mom наложила ограничения доступа для Son.

Рассмотрим код:

```
#include <iostream>
struct Base
{
    virtual void f() const { std::cout << 1; }
};
struct Derived: public Base
{
    virtual void f() /*override*/ { std::cout << 2; }
};
int main()
{
    Derived d;
    d.f();
    Base& b = d;
    b.f();
}
```

Ответьте на вопросы:

1) Скомпилируется ли этот код? Если нет, то почему, а если да, то что выведется на экран и почему?

2) Что изменится, если раскомментировать слово `override`? Поясните свой ответ.

1) Все скомпилируется и выведется 21. Потому что это две разных виртуальных функции (у них разная сигнатура), а значит, вторая не продолжает другую.

2) Если написать `override`, то будет СЕ, т.к. вторая `f` на самом деле никого не переопределяет.

Рассмотрим код:

```
#include <iostream>
struct Granny
{
    /*virtual*/ void foo() { std::cout << "Granny\n"; }
};
struct Mother: public Granny
{
    void foo() { std::cout << "Mother\n"; }
};
struct Father: public Granny
{
    void foo() { std::cout << "Father\n"; }
};
struct Son: public Mother, public Father
{};
int main()
{
    Son s;
    Mother& m = s;
    dynamic_cast<Father&>(m).foo(); // 1
    dynamic_cast<Son&>(m).foo(); // 2
}
```

Оказалось, что данный код не компилируется: ошибки компиляции возникают в обеих выделенных строках. Ответьте на вопросы:

- 1) Почему возникают ошибки компиляции в данном коде?
- 2) Поменяется ли что-нибудь, если раскомментировать слово `virtual`? Исчезнут ли **ошибки, появятся ли другие вместо них? Поясните подробно.**

1) Будет СЕ оба раза при попытке сделать `dynamic_cast`. Потому что он применим только к полиморфным типам, а это не наш случай.

2) Если раскомментировать `virtual`, то СЕ в строке 1 пропадет, а в строке 2 все еще будет СЕ. Но на этот раз по причине неоднозначности, т.к. у `Son` есть две версии `foo` (от мамы и от папы), непонятно какую вызвать.

Рассмотрим следующий код:

```
#include <iostream>
template<int N>
struct Sum
{
    const int value = N + Sum<N-1>::value;
};
struct Sum<0>
{
    const int value = 0;
};
int main()
{
    std::cout << Sum<100>::value;
}
```

Какие ошибки компиляции вызовет данный код?

Объясните, что нужно исправить в реализации метафункции Sum, чтобы эта программа успешно компилировалась и выполнялась без ошибок, а на экран бы выводилось значение **суммы натуральных чисел от 1 до 100**.

Тут есть две проблемы (за правильное исправление каждой по 1 баллу).

1) Забыто слово `static`. Значения `value` должны быть статическими полями, т.к. мы не собираемся создавать объектов типа `Sum<N>`.

2) Объявление специализации `Sum<0>` должно начинаться с `template<>`.

Рассмотрим программу:

```
#include <iostream>
template <typename T>
struct S
{
    T x;
    S(T x): x(x) {}
};
int main()
{
    S<int> s1(5);
    S<double> s2(3.14);
    s1 = s2;
    std::cout << s1.x << s2.x;
}
```

1) Какая ошибка произойдет при попытке ее скомпилировать и запустить? Объясните, почему она произойдет.

2) Не меняя код функции main, допишите что-нибудь в тело структуры S, так, чтобы код **успешно компилировался и выполнялся без ошибок.**

1) Будет СЕ в строке `s1 = s2`, т.к. `S<int>` и `S<double>` - это разные и не связанные между собой типы, нет способа присвоить один другому.

2) Надо дописать, например, такой оператор присваивания:

```
template <typename U>
void operator=(S<U> other) { x = other.x; }
```

Возвращаемый тип можно сделать void, а можно и S&, для целей данной задачи это не важно.